NEOLABS

SECURITY LABS · SOC LEVEL 1

# Wazuh Deployment and Investigation Guide

Architecture, dashboard workflow, workstation setup, recovery and troubleshooting

**Version:** week1-prod-234

**Publication date:** 2026-10-29

**Classification:** Student Training Material

Authorised synthetic training use only.

# Contents

1. Wazuh SOC Level 1 Handbook — Module 1

2. Wazuh Dashboard Tutorial 01 — Orientation and First Investigation

3. NeoLabs SOC L1 Workstation Compatibility

4. NeoLabs SOC L1 Backup and Recovery

5. NeoLabs Wazuh Setup and Troubleshooting Guide

6. NeoLabs Student Wazuh Stack

Source: docs/wazuh-handbook/01-wazuh-architecture-and-neolabs-deployment.md

# Wazuh SOC Level 1 Handbook — Module 1

# Architecture, Data Flow and the NeoLabs Student Deployment

**NeoLabs tested baseline:** Wazuh 4.14.7
**Module version:** 1.0
**Reconciled:** 14 August 2026
**Audience:** Beginner-to-intermediate SOC Level 1 interns

> Wazuh is not one program running in one window. It is a set of components that collect, analyse, store and display security data. A SOC analyst should be able to identify which stage produced—or failed to produce—the evidence they are looking for.

For the current programme/runtime summary, also read `../../PROGRAMME_CURRENT_STATE.md`.

---

## Learning objectives

By the end of this module, you should be able to:

- explain the roles of the Wazuh manager, Filebeat, indexer and dashboard;
- explain how the NeoLabs VCC telemetry feed differs from a normal Wazuh endpoint agent;
- describe how a VCC event becomes a searchable Wazuh alert;
- distinguish source `event_time`, replay/ingestion metadata and Wazuh index time;
- explain why an empty dashboard is not automatically evidence that no event occurred;
- use the current NeoLabs one-click Windows startup and Doctor workflow;
- explain the Night Watch and Telemetry Health views;
- identify the student/operator trust boundary and pod-isolation controls;
- perform a safe first-line data-flow check without exposing credentials.

---

# 1. What Wazuh is

Wazuh is an open-source security platform that can collect and analyse operating-system, application, network, cloud and security telemetry. In a typical deployment it combines endpoint agents/collectors, a Wazuh server or manager, Filebeat, a Wazuh indexer and a Wazuh dashboard.

For SOC work, think of the stack as a pipeline:

```
source event
→ collection
→ parsing/decoding
→ rule evaluation
→ alert creation
→ forwarding
→ indexing
→ search/dashboard
```

A failure at one stage can make later stages look empty even when earlier stages are healthy. That is why NeoLabs troubleshooting verifies the full chain rather than only checking that Docker containers are running.

---

# 2. Core Wazuh components

## 2.1 Wazuh agent

A Wazuh agent is software installed on a monitored endpoint. It can collect operating-system/application logs, Windows Event channels, file-integrity activity, system inventory and other endpoint security information.

### NeoLabs VCC boundary

SOC interns do **not** install or administer Wazuh agents inside VCC application pods. The VCC remains operator-managed. The programme exports approved synthetic pod telemetry through a separate protected NeoLabs feed.

Therefore:

- absence of a pod-named Wazuh agent does not mean the VCC feed is absent;
- the student does not receive pod SSH/container access merely because Wazuh can see pod telemetry;
- student pod scope is controlled by the NeoLabs access service, not by a local Wazuh agent selector.

## 2.2 Wazuh manager

The Wazuh manager analyses received events. Important responsibilities include:

- reading configured log sources;
- decoding/parsing events;
- applying rules and correlations;
- writing alert records;
- coordinating supported Wazuh functions and API access.

The NeoLabs Compose service is:

```
wazuh.manager
```

The current manager configuration reads the shared NeoLabs VCC NDJSON stream at:

```
/var/ossec/logs/vcc/vcc-events.ndjson
```

with JSON log format.

## 2.3 Filebeat

Filebeat forwards Wazuh alerts from the manager toward the indexer.

This creates an important troubleshooting distinction:

```
manager parsed/matched the event
but
Filebeat/indexer did not make it searchable
```

That situation is different from a collector failure or a decoder/rule failure.

## 2.4 Wazuh indexer

The Wazuh indexer is the search/storage layer based on OpenSearch technology. It stores documents, mappings and searchable indices.

The NeoLabs Compose service is:

```
wazuh.indexer
```

Week 1 investigation primarily uses:

```
wazuh-alerts-*
```

Normal NeoLabs VCC baseline events are deliberately eligible to become searchable alerts so Operation Night Watch can show normal activity, not only high-severity detections.

## 2.5 Wazuh dashboard

The Wazuh dashboard provides the analyst interface for search, Threat Hunting/Discover, saved objects, dashboards and Wazuh modules.

The NeoLabs service is:

```
wazuh.dashboard
```

The normal student profile binds the dashboard to loopback:

```
https://127.0.0.1:8443
```

The dashboard, Wazuh API and indexer must not be exposed publicly merely to simplify student access.

---

# 3. The current NeoLabs student topology




## Persistence

Docker named volumes retain manager/indexer/dashboard state across ordinary stops. A normal `git pull` or stack restart does not require students to delete Wazuh volumes or regenerate credentials.

A destructive reset is a separate local-only operation and is **not** normal troubleshooting.

---

# 4. How VCC telemetry reaches the student

The programme separates **telemetry access** from **infrastructure access**.

A SOC intern receives:

- the SOC toolkit;
- a private NeoLabs Access Code;
- the server-assigned pod/track/scenario context;
- a local Wazuh workstation;
- only the synthetic telemetry/evidence authorised for that assignment.

The intern does **not** receive:

- EC2 or pod shell access;
- database/container administration;
- broad AWS credentials;
- another pod's telemetry;
- mentor ground truth;
- a client-side control that authorises a different pod.

### Server-authoritative scope

The student may type a pod number during login as confirmation, but the server-side assignment is authoritative. A local edit cannot turn a student into another pod/track.

Both live and replay paths validate the assigned scope. Replay additionally validates that records are synthetic, belong to the assigned pod and contain required event fields before they are appended to the local Wazuh telemetry stream.

---

# 5. LIVE versus REPLAY

The VCC uses a cost-aware hybrid runtime. The main VCC EC2 does not have to remain on for every hour of an investigation.

The same student access flow can represent different authorised SOC states:

- **LIVE** — protected live pod telemetry during approved interactive windows;
- **REPLAY** — archived telemetry for the assigned pod/scenario;
- **CLOUD_LIVE / ENDPOINT_LIVE** — scenario-specific cloud/endpoint evidence surfaces where applicable;
- **OFFLINE/no surface** — the toolkit must not pretend data is available.

### Event time during replay

Replay preserves the original source:

```
event_time
```

and adds replay metadata separately.

For an incident timeline, order events by original event time unless the assignment specifically asks you to analyse collection/replay delay.

---

# 6. How Wazuh analyses a NeoLabs event

A simplified event path is:

```
NDJSON event
→ JSON decoding
→ NeoLabs base rule
→ child/correlation rules where applicable
→ alert record
→ Filebeat
→ Wazuh indexer
→ dashboard search
```

A synthetic VCC event may contain fields such as:

```
{
  "schema_version": "1.0",
  "event_id": "evt-auth-0009",
  "event_time": "2026-08-14T09:18:07Z",
  "pod_id": "pod-03",
  "event_type": "authentication",
  "action": "login",
  "outcome": "success",
  "user": "student.synthetic",
  "source_ip": "203.0.113.44",
  "session_id": "sess-902",
  "correlation_id": "corr-902",
  "synthetic": true
}
```

Wazuh may expose NeoLabs dynamic fields under `data.*`. In the current mapping, the NeoLabs user identity is commonly visible as `data.dstuser` in the saved Week 1 view.

---

# 7. NeoLabs custom rules

The current NeoLabs VCC rule set includes training rules for:

- normal VCC baseline events;
- authentication failures;
- repeated authentication failures;
- successful authentication;
- success after earlier failures;
- sensitive account changes;
- protected authorisation denials;
- telemetry collection/parser/visibility problems.

The base VCC event rule is intentionally searchable at a non-zero alert level so normal Week 1 activity can appear in `wazuh-alerts-*`.

Telemetry-health problems are represented around rule:

```
100150
```

A Wazuh rule match means the configured condition was observed. It does **not** automatically prove malicious intent or impact.

---

# 8. Current Windows setup and startup

The normal student workflow has changed from the older manual enrolment sequence.

## First/current startup

From the latest toolkit checkout:

1. Start Docker Desktop with WSL2 integration.
2. Double-click:

```
START-NEOLABS-SOC.cmd
```

The launcher automatically:

1. verifies WSL2/toolkit prerequisites;
2. performs first-time Wazuh preparation only when needed;
3. preserves an existing `.env` and Wazuh data;

4. reuses a valid NeoLabs session or asks for assigned pod + private Access Code;
5. connects to the current authorised LIVE/REPLAY surface;
6. starts/waits for manager, indexer, dashboard and telemetry collector;
7. reloads the current NeoLabs rule file in the manager;
8. proves an assigned-pod VCC event is searchable in `wazuh-alerts-*`;
9. reports latest-event freshness;
10. applies/checks local alert-index retention/disk safety;
11. provisions Night Watch/Telemetry Health saved objects where supported;
12. copies the local Wazuh `admin` password to the Windows clipboard without printing it; and
13. opens the local dashboard.

Do not begin cohort analysis until the launcher displays:

```
SOC WORKSTATION READY
```

## No global CLI/manual gateway requirement

For the normal Windows programme path, students do **not** need to:

- run `python -m pip install -e .`;
- add Python Scripts to Windows PATH;
- copy a private/manual gateway URL;
- manually enrol with a token file;
- manually select another telemetry target.

Advanced/manual scripts still exist for troubleshooting and non-Windows operation, but they are not the primary student onboarding sequence.

---

# 9. Wazuh dashboard login

After READY, the normal dashboard is:

```
https://127.0.0.1:8443
```

Use:

```
Username: admin
Password: the locally generated WAZUH_INDEXER_PASSWORD
```

On Windows, the launcher copies the password to the clipboard without displaying it. Press `Ctrl+V` at the login page, then replace the clipboard contents with non-sensitive text after login.

The following are **not** the human dashboard password:

- `WAZUH_API_PASSWORD` — internal Wazuh API/service communication;
- `WAZUH_DASHBOARD_PASSWORD` — internal dashboard service account.

Never paste the local Wazuh password into an assignment, screenshot, Slack post or Git commit.

---

# 10. Night Watch and Telemetry Health views

## NeoLabs — Operation Night Watch

The toolkit attempts to provision a Week 1 pod-scoped saved view/dashboard containing important investigation fields such as:

- `data.pod_id`;
- `data.event_type`;
- synthetic user identity;
- `data.source_ip`;
- `data.outcome`;
- `data.correlation_id`;
- Wazuh `rule.id`, `rule.level`, `rule.description`;
- original `data.event_time`.

Use it to establish normal authentication/application/API behaviour and build reusable Week 1 searches.

## NeoLabs — Telemetry Health

This view focuses on telemetry-quality problems, especially rule `100150`, so students can distinguish “no suspicious event” from “the collection/search pipeline is unhealthy.”

Saved-object provisioning is fail-soft. If an import is unavailable on a local dashboard instance, Threat Hunting/Discover remains the supported fallback.

---

# 11. Freshness and local retention

The current toolkit reports the newest assigned-pod event indexed in Wazuh.

Default freshness warning:

```
90 minutes
```

This is a health signal—not proof that an attack should happen every 90 minutes.

Current local alert-index defaults:

```
wazuh-alerts-* retention: 30 days
filesystem warning:       85%
filesystem critical:      92%
```

This retention applies only to the student's local Wazuh alert indices. It does **not** delete VCC server-side telemetry archives/evidence and does not force-overwrite unrelated existing index-management policies.

---

# 12. NeoLabs Doctor

On Windows, double-click:

```
CHECK-NEOLABS-SOC.cmd
```

or run:

```
.\neolabs.cmd doctor
```

Doctor checks the pipeline in order:

```
1. NeoLabs authentication
2. current LIVE/REPLAY telemetry surface
3. raw VCC event file
4. Wazuh rule engine
5. Filebeat
6. Wazuh indexer
7. Wazuh dashboard
```

It also reports telemetry freshness and local index/disk state.

This is the preferred first troubleshooting tool because it prevents the common mistake of changing Wazuh rules when the real failure is earlier in the pipeline.

---

# 13. Why an empty dashboard may be misleading

Before writing “no events were found,” confirm:

1. Doctor/indexer health passes;
2. the latest VCC event is reasonably fresh for the current assignment/window;
3. the filter uses the assigned `pod_id`;
4. the correct `wazuh-alerts-*` data view is selected;
5. the time range includes the original event times;
6. Telemetry Health does not show a collection/parser/visibility problem.

A zero-result query becomes useful evidence only after these assumptions are checked.

---

# 14. Alerts versus archives

`wazuh-alerts-*` contains searchable alert documents. The current NeoLabs Week 1 base rule ensures normal approved VCC baseline events can be represented there.

Wazuh archive indexing, when enabled, can provide additional events that did not become alerts. Archive indexing may not be available on every student workstation because it increases storage/resource/privacy requirements.

Do not assume `wazuh-archives-*` exists. If the assignment needs evidence that is not available in alerts, record the visibility limitation and use only mentor-approved evidence sources.

---

# 15. Network exposure and security boundaries

Common Wazuh ports can include agent communication/enrolment, server API, indexer and dashboard ports. In the NeoLabs student workstation:

- the dashboard is loopback-only by default;
- the Wazuh API/indexer are not intentionally published for student network access;
- internal components communicate on the Compose network;
- VCC telemetry scope is server-managed;
- students must not expose the local stack publicly for convenience.

The fact that Wazuh is a security tool does not make an insecure Wazuh deployment acceptable.

---

# 16. Safe manual checks

Advanced/manual checks remain available from the toolkit root, for example:

```
bash wazuh-stack/scripts/compatibility-check.sh
bash wazuh-stack/scripts/health-check.sh
bash wazuh-stack/scripts/verify-telemetry-pipeline.sh --wait 180
bash wazuh-stack/scripts/telemetry-freshness.sh
bash wazuh-stack/scripts/doctor.sh
```

Use `wazuh-logtest` through the supported verifier/Doctor workflow when checking decoding/rules. Do not broadly raise rule levels, disable TLS, publish indexer/API ports or delete volumes simply to make a dashboard show data.

---

# 17. Evidence and confidentiality

Useful Week 1 evidence includes:

- Wazuh event/rule IDs;
- assigned pod and explicit time range;
- relevant source event fields;
- saved/reproducible filters;
- original event-time timeline;

- telemetry-health/freshness notes;
- redacted screenshots.

Never submit:

- NeoLabs Access Code;
- Wazuh local password;
- session token;
- signed private URL;
- private key/certificate contents;
- AWS credentials;
- another pod's data;
- real customer/production information.

---

# 18. Week 1 practical checklist

For Operation Night Watch:

1. Start with `START-NEOLABS-SOC.cmd`.
2. Wait for `SOC WORKSTATION READY`.
3. Log in to the local Wazuh dashboard as `admin`.
4. Open NeoLabs — Operation Night Watch or Threat Hunting.
5. Confirm your assigned pod filter.
6. Record latest-event freshness and inspect Telemetry Health.
7. Find normal authentication and application/API activity.
8. Pivot using identity/source/session/correlation fields.
9. Save/reuse at least three baseline searches.
10. Build a short original-event-time timeline.
11. Record one visibility limitation.
12. Submit only redacted evidence through the central assignments repository.

---

# 19. Completion standard

You understand the NeoLabs Wazuh architecture when you can answer all of these without guessing:

- Where did this event originate?
- How did it reach the local telemetry file?
- Did Wazuh decode and match a rule?
- Did Filebeat/indexer make it searchable?

- Am I using the correct assigned pod and time range?
- Is the newest telemetry fresh/healthy?
- Is this rule an observation or proof of compromise?
- What can the current telemetry prove, and what can it not prove?

That pipeline awareness is one of the most important habits of a Level 1 SOC analyst.

Source: docs/dashboard-tutorials/01-orientation-and-alert-investigation.md

# Wazuh Dashboard Tutorial 01 — Orientation and First Investigation

**NeoLabs tested baseline:** Wazuh 4.14.7
**Reconciled:** 2026-08-14
**Audience:** SOC Level 1 interns

Dashboard labels can vary by release. Current startup behaviour is documented in `../../PROGRAMME_CURRENT_STATE.md`.

## Before opening the dashboard

Start through the platform root launcher:

```
Windows: START-NEOLABS-SOC.cmd
Linux:   ./start-neolabs-soc.sh
```

On a first Linux run, `bash start-neolabs-soc.sh` is also valid.

Wait for `SOC WORKSTATION READY`. This means the toolkit verified service health **and** that assigned-pod VCC telemetry is searchable in the local Wazuh indexer.

If anything looks wrong:

```
Windows: START-NEOLABS-SOC.cmd doctor
Linux:   ./start-neolabs-soc.sh doctor
```

Normal dashboard URL:

```
https://127.0.0.1:8443
```

Login username is `admin`. Windows copies the locally generated password to the clipboard without printing it. A headless Linux host prints secure port-forward guidance rather than exposing the dashboard publicly.

# Current NeoLabs investigation views

### NeoLabs — Operation Night Watch

Use this preconfigured pod-scoped saved view/dashboard for Week 1 when provisioning succeeds. It focuses on:

- assigned `pod_id`;
- `event_type`;
- NeoLabs user identity (`data.dstuser` in the current Wazuh mapping);
- `source_ip`;
- `outcome`;
- `correlation_id`;
- Wazuh rule ID/level/description;
- original source `event_time`.

### NeoLabs — Telemetry Health

Use this saved view for collector/parser/visibility problems, especially NeoLabs rule `100150`.

Saved-object provisioning is fail-soft. If these views are unavailable, use **Threat Hunting/ Discover** against `wazuh-alerts-*`; do not reset Wazuh because a saved view did not import.

# Alerts versus raw/archive data

Week 1 begins with `wazuh-alerts-*`. Normal VCC baseline events are deliberately eligible for searchable alerts so interns can see normal Night Watch activity, not only higher-severity detections.

`wazuh-archives-*` may not be enabled on every student workstation. If archive data is unavailable, state the visibility limitation rather than assuming the event never existed.

# Time is evidence

For assignment evidence:

1. locate the relevant event(s);
2. choose an absolute UTC range around their **original event time**;
3. record that range in the query journal;
4. distinguish source `event_time` from replay/ingest/indexed timestamp.

Replay preserves original `event_time` and stores replay metadata separately.

## First Week 1 investigation flow

1. Open **NeoLabs — Operation Night Watch** or Threat Hunting.
2. Confirm the assigned `pod_id`. If another pod appears, stop and contact a mentor.
3. Check latest-event freshness reported by the launcher/Doctor.
4. Inspect any Telemetry Health warning before interpreting missing data.
5. Set an absolute time range around Week 1 events.
6. Expand a record and capture `event_id`, original `event_time`, `event_type`, identity, source, outcome, session/correlation IDs and Wazuh rule metadata where available.
7. Find normal successful authentication and an ordinary failed authentication if present.
8. Pivot by identity → source IP → session/correlation ID to related application/API activity.
9. Identify storage/other approved telemetry visible to the pod.
10. Save/reuse at least three baseline filters/searches.
11. Build a short original-event-time timeline.
12. Record one visibility gap.

Week 1 is baseline building. “Unusual” does not automatically mean “malicious.”

## Useful NeoLabs rule families

Current training rules include normal baseline events plus authentication failure/repeated failure, authentication success/success-after-failures, sensitive account change, authorisation denial and telemetry-health alerts.

Rule IDs/severity are pivots. Inspect underlying event fields/context before disposition.

## Filters and pivots

A practical pivot sequence is:

```
pod
→ identity
→ source_ip
→ session_id / correlation_id
→ related application/auth/account activity
→ telemetry health for the same window
```

Do not reproduce denied/cross-pod requests merely to create evidence.

## WQL/search-bar caution

Wazuh Query Language, OpenSearch/Discover query syntax and UI filter pills are not interchangeable everywhere. Prefer UI field filters when following this tutorial. If using a query language, record which view/search bar accepted it so another analyst can reproduce the result.

## Zero results are not automatically evidence of absence

Before writing “no events found”:

- confirm Doctor/indexer PASS;
- confirm latest VCC event freshness;
- confirm the correct data view/index pattern;
- confirm the assigned pod filter;
- confirm absolute time range/time zone;
- inspect Telemetry Health / rule `100150`;
- state missing archive/source coverage if relevant.

## Evidence capture

A useful screenshot should show enough context to prove the claim: view/search name, time range, active filters and relevant event/rule fields. Redact the local Wazuh password, NeoLabs Access Code/session, signed URLs, private keys/certificates and unrelated personal data.

## Dashboard/agent distinction

The NeoLabs VCC pod feed is consumed as a protected telemetry stream/local file path; it is not permission to install/manage an agent inside a VCC pod. Absence of a pod-named Wazuh agent does not mean the VCC feed is absent.

## When to use Doctor instead of changing settings

Use the platform launcher `doctor` action before editing dashboard/indexer/manager settings. Doctor identifies whether the break is at authentication, replay/live access, raw event, rule engine, Filebeat, indexer or dashboard.

Do not weaken TLS, expose indexer/API ports, disable rules broadly or delete local volumes simply to make the dashboard show data.

## Completion check

You can complete this tutorial when you can explain the assigned pod, current data source/time range, one normal authentication sequence, one application/API pivot, the meaning of the relevant Wazuh rule, one saved/reproducible filter and one visibility limitation—without confusing replay/index time with source event time.

Source: docs/setup/WORKSTATION_COMPATIBILITY.md

# NeoLabs SOC L1 Workstation Compatibility

For current startup behaviour see `../../PROGRAMME_CURRENT_STATE.md`.

## Supported baseline

| Requirement | Hard floor | Preferred minimum | Recommended |
|---|---|---|---|
| CPU architecture | 64-bit x86_64 or arm64 | 64-bit x86_64 | x86_64 |
| CPU capacity | 4 logical cores | 4 | 6+ |
| Memory visible to Linux/WSL2 | 7 GiB | 8 GiB | 12–16 GiB |
| Free disk | 25 GiB | 25 GiB | 50 GiB |
| Container runtime | Docker Engine/Desktop | Docker Engine/Desktop | current supported release |
| Compose | v2 | v2 | current supported v2 |
| `vm.max_map_count` | 262144 | 262144 | >=262144 |

Seven GiB is the constrained Week 1 memory floor. The launchers can install software and configure a kernel setting; they cannot fix insufficient RAM, CPU, disk capacity, blocked virtualisation or organisation policy.

## Physical Windows 10/11 + WSL2

Use only:

```
START-NEOLABS-SOC.cmd
```

The Windows path is supported for **physical Windows 10/11 workstations**. Before attempting WSL/Docker setup, the internal bootstrap checks Windows OS type and common VM/VPS hardware identifiers. It fails early when it detects Windows Server or a Windows VM/VPS guest and directs the intern to an Ubuntu/Debian VPS instead.

On a supported Windows workstation the launcher owns:

- WSL2 detection/bootstrap;
- an existing current WSL2 distribution, or Ubuntu installation only when no Linux distro exists;

- Docker Desktop detection/installation/startup;
- Linux container engine + WSL integration verification;
- missing Linux packages inside WSL;
- `vm.max_map_count` adjustment through the WSL root account;
- Wazuh first-run preparation and subsequent startup.

Ubuntu is not mandatory when the intern already has a suitable Kali, Debian, Ubuntu or other current WSL2 distro. Git Bash/MSYS/Cygwin is not a supported Wazuh runtime.

Windows can require one restart after enabling WSL or one initial launch of a newly installed Linux distro to create its Linux user. Rerun the same root CMD afterward.

If WSL2 cannot start on a **physical** Windows computer because firmware virtualisation is disabled, the intern must enable Intel VT-x/AMD-V/virtualisation in the computer BIOS/UEFI. The launcher cannot change a physical machine's BIOS/UEFI setting itself.

## Ubuntu / Debian Linux

Use:

```
bash start-neolabs-soc.sh
```

Run it as the normal Linux account, **not** with `sudo` in front of the whole command. The launcher invokes administrator privileges itself only for:

- package/repository installation;
- Docker service/group setup;
- `vm.max_map_count` configuration/persistence.

When Docker is absent on Ubuntu/Debian, the launcher installs Docker Engine + Compose v2 and then gives the normal user Docker access. Wazuh runtime commands themselves should then run without `sudo`.

## VPS / remote server policy

For an intern who chooses a VPS or remote server, the supported platform is **Ubuntu or Debian Linux only**.

Recommended images:

- Ubuntu 24.04 LTS;
- Ubuntu 22.04 LTS;
- a current Debian release.

Do not provision Windows Server or a Windows desktop VM/VPS for the NeoLabs SOC workstation. WSL2 inside a Windows guest depends on nested virtualisation being exposed by the host hypervisor/provider, which cannot be enabled by the NeoLabs script from inside the guest. The programme avoids that dependency by running Docker Engine/Wazuh directly on Linux VPS hosts.

On a VPS:

```
bash start-neolabs-soc.sh
```

The Wazuh dashboard remains loopback-only. The launcher prints an SSH port-forward example for headless remote access.

## Other Linux distributions

The root Bash launcher can operate when its prerequisites already exist. Automatic Docker installation is intentionally bounded to Ubuntu/Debian. On another distribution, install Docker Engine + Compose v2 through that distribution's supported method, then rerun the same launcher. For programme-provisioned VPS systems, use Ubuntu/Debian rather than another distribution.

## Headless Linux server

Wazuh still binds the dashboard to loopback. If the server has no desktop/browser, the launcher prints a secure SSH local-port-forward example. Do not change the dashboard binding to `0.0.0.0` merely to make remote access easier.

## Unsupported configurations

- Windows Server as the SOC workstation;
- Windows VM/VPS guests as the SOC workstation;
- VPS/remote SOC workstations using a non-approved OS instead of Ubuntu/Debian;
- 32-bit operating systems;
- Docker Compose v1/legacy Docker Toolbox;
- less than 7 GiB Linux/WSL2-visible memory;
- systems where local Docker cannot be used;
- Git Bash/MSYS/Cygwin as the Wazuh runtime;
- public exposure of the local Wazuh administrative services;
- production/company shared infrastructure used as an internship target/workstation without explicit approval.

## Diagnostics

Windows physical workstation:

```
START-NEOLABS-SOC.cmd doctor
```

Linux / Ubuntu / VPS:

```
./start-neolabs-soc.sh doctor
```

Mentors/advanced troubleshooting may still run `bash wazuh-stack/scripts/compatibility-check.sh`, but that internal check is diagnostic only. Students should not repair its failures by guessing where to add `sudo`; the platform launcher owns supported setup changes.

Source: docs/setup/BACKUP_AND_RECOVERY.md

# NeoLabs SOC L1 Backup and Recovery

## Purpose

The student Wazuh environment contains locally generated investigation data, dashboards, indexer data and manager state. A workstation failure should not require a learner to rebuild every investigation from the beginning.

The toolkit therefore provides a guarded Docker-volume backup and restore process. It is designed for local training data, not for production disaster recovery.

## Security boundary

A toolkit backup deliberately excludes:

- `.env`;
- VCC bootstrap tokens;
- client private keys;
- client certificates and CA trust files;
- enrolment responses;
- private VCC endpoints;
- production data or credentials.

After restoring, the learner must complete a fresh operator-approved enrolment. This prevents copied backups from becoming reusable VCC access packages.

## Create a backup

From the repository root:

```
bash wazuh-stack/scripts/backup.sh
```

The script performs the following actions:

1. confirms that Docker, Compose and the local `.env` file are available;
2. discovers the named volumes declared by the Wazuh Compose file;
3. stops the running stack to obtain a consistent point-in-time snapshot;
4. creates one compressed archive per existing volume;
5. calculates SHA-256 checksums;
6. writes a manifest and recovery notice;
7. restarts the stack unless `--no-restart` was supplied.

Use a specific destination when required:

```
bash wazuh-stack/scripts/backup.sh --output /secure/local/path/neolabs-backup
```

Keep the backup on encrypted, personally controlled storage. Do not upload it to a public repository or share it in Slack or WhatsApp.

## Verify a backup

Always verify a backup before relying on it:

```
bash wazuh-stack/scripts/verify-backup.sh /path/to/backup
```

Verification checks the manifest version, checksum file, archive readability, presence of at least one volume and absence of common credential filenames.

## Restore a backup

A restore replaces the contents of the matching Wazuh volumes. Stop the stack and confirm the backup path before continuing:

```
bash wazuh-stack/scripts/stop.sh
bash wazuh-stack/scripts/restore.sh /path/to/backup --force
```

The restore script:

- verifies checksums before modifying volumes;
- accepts only volume names declared in the current Compose file;
- refuses to run while stack services are active;
- recreates and repopulates the declared volumes;
- removes stale local VCC collector scope and cursor state;
- does not restore enrolment credentials.

After restoration:

```
bash wazuh-stack/scripts/compatibility-check.sh
bash wazuh-stack/scripts/start.sh
bash wazuh-stack/scripts/health-check.sh
```

Then request a new enrolment token from the programme operator and complete the normal enrolment workflow.

## Rehearsal

The repository CI performs a synthetic Docker-volume recovery rehearsal using:

```
bash wazuh-stack/tests/backup-restore-rehearsal.sh
```

The rehearsal creates a temporary volume, writes a synthetic marker, archives it, verifies its checksum, deletes and recreates the volume, restores the archive and confirms that the marker survived. It never reads the learner's Wazuh data.

## Recovery limitations

A successful archive verification does not prove that every future Wazuh release can read data created by an older release. Restore into the same pinned toolkit version first. Complete a documented Wazuh upgrade only after the restored stack is healthy.

Backups are not a substitute for GitHub submissions. Investigation reports, evidence logs and query journals should still be committed to the approved assignment repository after removing credentials, private URLs and sensitive evidence.

Source: troubleshooting/WAZUH_SETUP_AND_TROUBLESHOOTING_GUIDE.md

# NeoLabs Wazuh Setup and Troubleshooting Guide

**Tested baseline:** Wazuh 4.14.7
**Reconciled:** 2026-08-14
**Scope:** learner-owned workstation + authorised synthetic VCC telemetry only

## 1. Start with the platform root launcher

Windows:

```
START-NEOLABS-SOC.cmd doctor
```

Linux/Ubuntu:

```
./start-neolabs-soc.sh doctor
```

Do not start troubleshooting by running random files in `wazuh-stack/scripts/` or by adding `sudo` to a command that failed. The root launchers own supported prerequisite installation and privilege changes; the lower-level Wazuh scripts are intentionally runtime/validation components.

## 2. Golden path

```
NeoLabs authentication / server assignment
→ current LIVE/REPLAY surface
→ raw assigned-pod VCC event file
→ Wazuh manager localfile input
→ NeoLabs Wazuh rules
→ Filebeat
→ Wazuh indexer (wazuh-alerts-*)
→ dashboard / saved views / time filters
```

`SOC WORKSTATION READY` is displayed only after assigned-pod telemetry is actually searchable in the indexer.

# 3. First-run prerequisite problems

### Windows

Rerun `START-NEOLABS-SOC.cmd` after resolving the exact state it reports. The launcher handles supported WSL2/Docker Desktop/bootstrap work. Legitimate one-time interruptions include a Windows restart after enabling WSL, launching a newly installed Linux distro once to create its Linux user, or resolving a Docker Desktop first-run/virtualisation/update prompt.

Do not run the Wazuh stack from Git Bash/MSYS/Cygwin.

### Linux / Ubuntu

Run the launcher as the normal user:

```
bash start-neolabs-soc.sh
```

Do **not** run `sudo ./start-neolabs-soc.sh`. The launcher invokes `sudo` itself only for package installation, Docker service/group setup and kernel configuration.

On Ubuntu/Debian the launcher can install Docker Engine + Compose v2 when missing. If it adds the user to the Docker group, it attempts to refresh that group context immediately; where the shell cannot do so, log out/in once and rerun the same root launcher.

# 4. Docker problems

Useful mentor/advanced checks:

```
docker --version
docker compose version
docker info
```

On Windows, Docker must be the Linux engine and reachable from the same WSL2 distro used by the toolkit. On native Linux, the normal user must be able to run `docker info` without prefixing every Wazuh command with `sudo`.

Never expose an unauthenticated Docker TCP socket to solve a permissions problem.

# 5. Memory, disk and indexer kernel requirement

Current workstation baseline:

- 7 GiB Linux/WSL-visible RAM hard floor;
- 8 GiB preferred;

- 12–16 GiB recommended;
- 25 GiB free disk hard floor; 50 GiB recommended;
- `vm.max_map_count >= 262144`.

The root launcher configures `vm.max_map_count` when it can. If an organisation/server policy blocks that administrator change, the host administrator must resolve the policy; do not edit Wazuh rules/Compose to bypass the indexer's requirement.

Local indexer filesystem warnings begin at 85% and become critical at 92%. Preserve required evidence before any mentor-approved cleanup; do not delete Wazuh volumes merely to clear a warning.

## 6. Local Wazuh configuration

The launchers generate `wazuh-stack/.env` only when it is absent and preserve an existing file. Do not regenerate local secrets because the repository was updated.

Human dashboard login:

```
Username: admin
Password source: local WAZUH_INDEXER_PASSWORD
```

`WAZUH_API_PASSWORD` and `WAZUH_DASHBOARD_PASSWORD` are internal service credentials, not the human login.

## 7. NeoLabs authentication/runtime failures

Use the root launcher actions:

```
Windows: START-NEOLABS-SOC.cmd status
         START-NEOLABS-SOC.cmd login

Linux:   ./start-neolabs-soc.sh status
         ./start-neolabs-soc.sh login
```

The pod entered during login is confirmation only; server assignment remains authoritative. Do not edit runtime state or a base URL to reach another pod.

## 8. Replay/live telemetry not arriving

Run Doctor, then verify the server state is an authorised SOC surface, raw telemetry contains synthetic assigned-pod records, replay validation did not reject pod/scenario fields, live enrolment is current when LIVE, and internet/TLS connectivity is available.

Replay preserves original `event_time`; replay time is not the incident sequence.

## 9. Raw events exist but Wazuh has no alerts

The manager must monitor `/var/ossec/logs/vcc/vcc-events.ndjson` as JSON and load `config/rules/neolabs_vcc_rules.xml`.

Normal startup restarts the manager after asserting the stack so current rules are loaded after `git pull`. The telemetry verifier uses `wazuh-logtest` as part of the rule-engine proof.

Do not broadly raise rule levels simply to make a dashboard busy.

## 10. Filebeat/indexer problems

If raw events/rule matching works but `wazuh-alerts-*` remains empty, Doctor separates Filebeat and indexer checks. Verify indexer health before changing mappings/policies. NeoLabs retention is deliberately scoped to local `wazuh-alerts-*` and does not force-overwrite unrelated policies.

## 11. Dashboard opens but looks empty

Before concluding “no events”:

1. confirm Doctor/indexer PASS;
2. check latest indexed VCC event freshness;
3. confirm assigned `pod_id` filter;
4. use `wazuh-alerts-*`, Threat Hunting or **NeoLabs — Operation Night Watch**;
5. set a correct absolute UTC range around original event times;
6. inspect **NeoLabs — Telemetry Health** / rule `100150`.

A zero-result query is useful evidence only after source health and time/filter assumptions are verified.

## 12. Saved Night Watch/Telemetry Health views missing

Saved-object provisioning is fail-soft. Healthy telemetry/indexing does not depend on the saved view import. Continue through Threat Hunting and report the view/import problem; do not reset Wazuh data merely because a saved object is absent.

## 13. Safe bounded repair

The root startup path permits one bounded local telemetry repair. It may reassert local services and, in replay mode, refetch the authorised replay state. It does not reset VCC server state, switch pods, delete Wazuh volumes or fetch another student's data.

If searchability still cannot be proven, stop relying on that workstation for assignment conclusions and send redacted Doctor output to the mentor.

## 14. Headless Linux dashboard access

The dashboard remains loopback-only. A headless server launcher prints an SSH local-forward example. Use that secure tunnel from the intern's own computer instead of changing the dashboard binding to a public address.

## 15. Advanced internal commands

Mentors may inspect `wazuh-stack/scripts/` directly for targeted diagnosis, backup/restore or CI rehearsal. These are not student setup steps.
`reset.sh --confirm-destroy-local-data` is destructive local recovery and is never a normal startup fix.

## 16. Evidence to send a mentor

Useful redacted evidence includes Doctor PASS/WARN/FAIL lines, current assigned pod/scenario/runtime state without credentials, latest event freshness, `docker compose ps` status, small relevant logs and event/rule IDs/screenshots.

Never send `.env`, admin password, Access Code, session token, private key/certificate contents, signed private URLs or another pod's data.

## 17. Stop conditions

Stop/escalate on cross-pod events/access, real personal/production data, exposed credentials/private keys, unexpected infrastructure access, external-target traffic or service instability. Do not weaken isolation/security controls to make a troubleshooting check pass.

Source: wazuh-stack/README.md

# NeoLabs Student Wazuh Stack

## Purpose

This directory contains the local Wazuh 4.14.7 manager, indexer, dashboard, telemetry collector, rules and maintenance scripts used by authorised NeoLabs SOC Level 1 interns.

**This directory is an implementation/runtime directory, not the normal student start location.**

## Student entry points

From the repository root:

### Windows

```
START-NEOLABS-SOC.cmd
```

### Linux / Ubuntu

```
bash start-neolabs-soc.sh
```

The platform launcher owns prerequisite installation, kernel configuration, first-run secret/config generation, Wazuh startup, NeoLabs authentication, telemetry connection, index verification and dashboard access. Students should not assemble a startup process by directly invoking scripts in this directory.

Diagnostics are also routed through the root launcher:

```
Windows: START-NEOLABS-SOC.cmd doctor
Linux:   ./start-neolabs-soc.sh doctor
```

## Internal telemetry path

```
VCC live/archive
→ authorised NeoLabs LIVE/REPLAY access
→ local vcc-events.ndjson
→ Wazuh manager JSON localfile input
→ NeoLabs VCC rules
→ Filebeat
→ Wazuh indexer wazuh-alerts-*
→ dashboard / Threat Hunting / saved views
```

Replay validates `synthetic=true`, assigned `pod_id`, required fields and scenario scope before append. Original `event_time` is preserved and replay metadata remains separate.

## READY contract

`SOC WORKSTATION READY` means a real synthetic event for the server-assigned pod has been processed and is searchable through the local Wazuh indexer. Container health alone is insufficient. If searchability initially fails, the root launcher permits only one bounded local repair attempt before refusing READY.

## Dashboard

Normal local URL:

```
https://127.0.0.1:8443
```

Human username is `admin`. The human password is the locally generated `WAZUH_INDEXER_PASSWORD` in private `.env`. Internal `wazuh-wui` / `kibanaserver` credentials are separate service credentials.

## Current NeoLabs rules

`config/rules/neolabs_vcc_rules.xml` includes training rules for normal VCC baseline events, authentication failures/repetition, successful authentication, success after failures, sensitive account changes, protected authorisation denials and telemetry-health problems (`100150`).

The base NeoLabs VCC rule produces searchable baseline documents, which is required for Operation Night Watch. Rule matches are investigation pivots, not automatic proof of malicious activity.

`start.sh` explicitly restarts the Wazuh manager after Compose assertion so updated bind-mounted NeoLabs rules are loaded after a toolkit update.

## Saved investigation views

Runtime provisioning attempts to create:

- **NeoLabs — Operation Night Watch** — assigned-pod baseline investigation view;
- **NeoLabs — Telemetry Health** — rule `100150` collection/parser/visibility troubleshooting view.

Saved-object provisioning is fail-soft; Threat Hunting remains available if local saved-object import fails.

## Freshness and local retention

- telemetry freshness warning default: **90 minutes**;
- local `wazuh-alerts-*` retention default: **30 days**;
- filesystem warning: **85%**;
- critical: **92%**.

These controls affect the student's local Wazuh data only; they do not delete VCC server-side telemetry/evidence.

## Workstation baseline

| Requirement | Hard floor | Preferred | Recommended |
|---|---|---|---|
| Linux/WSL2-visible memory | 7 GiB | 8 GiB | 12–16 GiB |
| CPU | 4 logical cores | 4 | 6+ |
| Free disk | 25 GiB | 25 GiB | 50 GiB |
| Compose | v2 | v2 | current v2 |
| `vm.max_map_count` | 262144 | 262144 | >=262144 |

The root launchers now own the supported privilege changes required to reach these software/kernel prerequisites. `compatibility-check.sh` and `preflight.sh` deliberately remain read-only validation scripts and should not contain scattered `sudo` commands.

## Internal/mentor operations

The scripts below remain available for CI, recovery and advanced mentor troubleshooting, but are not student-facing setup choices:

```
scripts/compatibility-check.sh
scripts/generate-local-secrets.sh
scripts/prepare-stack.sh
scripts/preflight.sh
scripts/start.sh
scripts/health-check.sh
scripts/verify-telemetry-pipeline.sh
scripts/repair-telemetry-pipeline.sh
scripts/telemetry-freshness.sh
scripts/doctor.sh
scripts/backup.sh / verify-backup.sh / restore.sh
scripts/stop.sh / reset.sh
```

Do not add `sudo` to those runtime commands simply because Docker/kernel setup failed. Correct the platform setup through the root launcher. `reset.sh` is destructive local recovery and is not normal Week 1 troubleshooting.

## Private files

Never commit/share `.env`, NeoLabs Access Codes/session files, VCC private keys/certificates/private signed URLs, local Wazuh passwords, enrolment/runtime state, generated backups or another pod's evidence.

## Network/security design

- Dashboard binds to loopback.
- Indexer and Wazuh API are not published for student network access.
- Internal components remain on the Compose internal network.
- Pod scope is server-managed.
- Collector/replay validation rejects wrong-pod/non-synthetic data.

Full troubleshooting guide:
`../troubleshooting/WAZUH_SETUP_AND_TROUBLESHOOTING_GUIDE.md`.